

Electric Vehicle Insurance Management System

Software Design & Project Documentation

Author: Poojith Sai Jasti · Version 1.0 (Draft for review) · October 8, 2026 · Classification: Non-Confidential

Document Control

Item	Detail
Project	Electric Vehicle (EV) Insurance Management System
Document type	Software design and project documentation
Author	Poojith Sai Jasti
Version / status	1.0 / Draft for review
Audience	Reviewers, mentors, developers, testers
Source of truth	Implemented system and the corrected presentation deck. Where the earlier draft conflicts with it, this document follows the implementation (see Appendix A).

Version	Date	Change
0.1	2026	Initial draft (microservices-based design)
1.0	Oct 8, 2026	Restructured and aligned to the implemented architecture, four-role model and EV product configuration

Conventions. Items marked (*Proposed*) are recommended additions that are not yet confirmed in the codebase. Items marked (*Confirm*) must be checked against the repository before submission. Items marked (*Target*) are goals, not measured results.

1. Executive Summary

The EV Insurance Management System is a role-based, full-stack web platform that digitizes the insurance lifecycle for electric vehicles: customer onboarding, EV registration, quotation, underwriting, policy issuance, claims, survey and settlement.

Generic motor-insurance workflows do not represent EV-specific risk, such as battery value, charging context and EV-specific covers. This system treats EV data and configurable EV coverages as part of the insurance domain. It is built with Angular and TailwindCSS on the front end, ASP.NET Core Web API with Entity Framework Core on the back end, SQL Server for persistence, and ASP.NET Identity with JWT for authentication and authorization.

Key outcomes

- One connected workflow from customer to settlement, with explicit states and role ownership.
- Admin-configurable products: policy types are mapped to coverage types, so product rules are not hard-coded.
- Four roles (Customer, Underwriter, Surveyor, Admin), each seeing only the operations relevant to them.

2. Scope, Assumptions and Constraints

2.1 In scope

- Registration, login, password management and role-based access.
- Customer profile, KYC document references and EV (vehicle) management.
- Quote generation, application, underwriting review, policy issuance, renewal and cancellation.
- Claim submission with evidence, surveyor assignment, investigation, recommendation and settlement.
- Admin management of employee requests, policy types, coverage types and product configuration.
- Document storage and notifications.

2.2 Out of scope (current release)

- Live payment gateway and real external APIs (payments are simulated or recorded only) (*Confirm*).
- Advanced actuarial EV risk scoring.
- Cloud deployment, containerization and CI/CD pipelines (listed as future scope).
- Asynchronous messaging between components.

2.3 Assumptions

- Users access the system through a modern desktop or mobile browser.
- A single organization operates the platform; there is no multi-tenancy.
- Employees (Underwriters, Surveyors) are onboarded through an Admin-approved request.

2.4 Constraints

- Technology is fixed to the .NET and Angular stack.
- There is no separate Vehicle Service; vehicle data is owned by the Customer domain.
- There is no Claims Officer role; the Surveyor performs verification and recommendation.

3. Domain Overview

Vehicle insurance protects against defined risks to a vehicle. The standard lifecycle is onboarding, vehicle details, quotation, policy issuance, servicing, claims and renewal.

EV insurance follows the same lifecycle but also considers battery capacity and value, charging equipment, vehicle type and EV-specific components.

Term	Meaning
Quote	Estimated premium for a vehicle and coverage selection, valid until a stated date
Application	Customer's request to buy a policy based on an accepted quote; reviewed by an Underwriter
Underwriting	Risk review that approves, rejects or adjusts the premium

Term	Meaning
Policy	The issued insurance contract with a start date, end date and status
Policy type	Product tier, for example Comprehensive EV, Premium EV, Third Party
Coverage type	A protection element, for example Battery, Flood, Theft, Roadside Assistance
Claim	Customer request for compensation after an incident
Survey	Surveyor's investigation and recommendation on a claim
IDV	Insured Declared Value of the vehicle
KYC	Know Your Customer identity verification

4. Actors and Business Journey

4.1 Roles and responsibilities

Role	Responsibilities
Customer	Register and log in; register EVs; request quotes; apply for a policy; view policies; renew; submit and track claims
Underwriter	Review applications; validate EV information; assess risk; modify premium when required; approve or reject
Surveyor	Receive assigned claims; review evidence; investigate the incident; record findings; submit recommendation
Admin	Approve employee requests; manage policy types and coverage types; configure products; manage configuration

4.2 Access matrix

C = create, R = read, U = update, D = delete, A = approve/decide, — = no access.

Resource	Customer	Underwriter	Surveyor	Admin
Own profile and EVs	CRUD	R	—	R
Quote	C, R (own)	R	—	R
Application	C, R (own)	R, U, A	—	R
Policy	R, renew, cancel (own)	R	—	R
Claim	C, R (own), withdraw	—	R (assigned), U	R
Survey report	R (status only)	—	C, R	R
Policy and coverage types	R	R	—	CRUD
Employee requests	—	—	—	A

4.3 End-to-end lifecycle

1. **Register:** the customer creates an account and registers an EV.
2. **Quote:** EV details and the selected product produce a premium estimate.
3. **Underwrite:** the Underwriter reviews the application and approves, rejects or adjusts the premium.
4. **Policy:** the approved application is issued and becomes active.
5. **Claim:** the customer submits a claim with evidence.
6. **Survey:** the Surveyor investigates and records findings and a recommendation.
7. **Settle:** the claim moves through the configured decision and settlement flow.

5. Requirements

5.1 Functional requirements

ID	Requirement	Role	Priority
FR-01	Customers can register and log in with email and password	Customer	High
FR-02	Employees can request access; Admin approves or rejects the request	Admin	High
FR-03	Users can reset and change passwords	All	Medium
FR-04	Customers can create and update a profile and submit KYC references	Customer	High
FR-05	Customers can register, update, list and remove EVs with battery and registration details	Customer	High
FR-06	The system generates a quote from vehicle, coverage, owner age and city	Customer	High
FR-07	Customers can apply for a policy from a valid quote	Customer	High
FR-08	Underwriters can review, adjust premium, approve or reject applications	Underwriter	High
FR-09	The system issues a policy with a unique policy number on approval and payment	System	High
FR-10	Customers can view, renew and cancel policies and download the certificate	Customer	High
FR-11	Customers can file a claim for an active policy and attach evidence	Customer	High
FR-12	Claims are assigned to a Surveyor, who submits findings and a recommendation	Surveyor	High
FR-13	Claims progress to settlement with an approved amount and payout status	System	High
FR-14	Admin can manage policy types, coverage types and their mapping	Admin	High

ID	Requirement	Role	Priority
FR-15	Users receive notifications for key events and can set preferences	All	Medium
FR-16	Access to every protected feature is enforced by role	All	High

5.2 Non-functional requirements

ID	Category	Requirement
NFR-01	Security	All protected endpoints require a valid JWT; passwords are stored only as salted hashes
NFR-02	Security	Server-side input validation on every request; HTTPS only
NFR-03	Performance	95% of standard API calls respond within 2 seconds under normal load (<i>Target</i>)
NFR-04	Reliability	State changes (policy issue, claim decision) execute inside a database transaction
NFR-05	Usability	Core journeys are completable in few steps; the UI is responsive on mobile and desktop
NFR-06	Maintainability	Clean Architecture layering; business logic lives in the service layer, not in controllers
NFR-07	Scalability	Stateless API so it can scale horizontally behind a load balancer
NFR-08	Auditability	Decisions record who acted and when (approvals, premium changes, claim decisions)
NFR-09	Data integrity	Foreign keys, unique constraints and migrations manage the schema

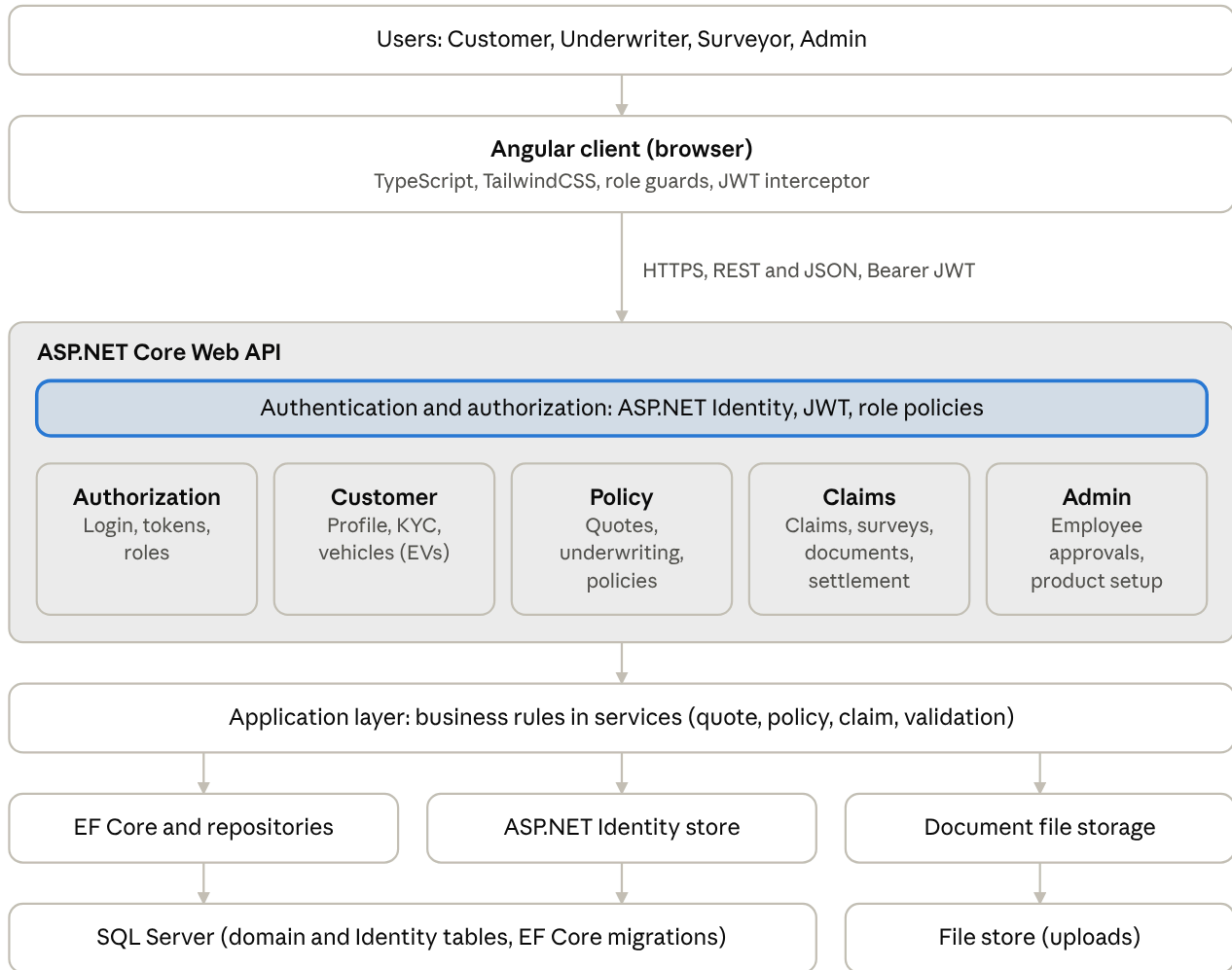
6. System Architecture

6.1 Architectural style

The system is a layered application following Clean Architecture. A single-page Angular client calls a REST API built on ASP.NET Core. The API is divided into five functional modules: Authorization, Customer, Policy, Claims and Admin. Business rules are implemented in a service layer, and data access uses Entity Framework Core against SQL Server.

Layer	Responsibility	Examples
Presentation (Angular)	UI, routing, guards, HTTP interceptors	Role dashboards, forms
API	HTTP endpoints, request validation, authorization attributes	Controllers
Application	Use cases and business rules	Quote, Policy, Claim services
Domain	Entities, enums, state rules	Quote, Policy, Claim
Infrastructure	EF Core, repositories, Identity, file storage	DbContext, migrations

Every request passes JWT authorization before it reaches one of five API modules



system architecture · 6 layers, 5 API modules

Figure 1. System architecture. The browser reaches the API only over HTTPS with a JWT, and the five modules share one application layer and one SQL Server database.

6.2 Module boundaries

Module	Owns
Authorization	Login, registration, tokens, roles
Customer	Customer profile, KYC, EVs
Policy	Quotes, applications, underwriting, policies
Claims	Claims, surveys, documents, settlement

Module	Owns
Admin	Employee requests, policy types, coverage types, configuration

Vehicles are intentionally part of the Customer module. There is no separate Vehicle Service.

6.3 Design decisions

#	Decision	Rationale
D1	Clean Architecture with a service layer	Testable business rules, replaceable infrastructure
D2	ASP.NET Identity with JWT	Proven, secure identity handling and stateless API
D3	No Claims Officer role	The Surveyor investigates and recommends, which keeps the workflow short
D4	Config-driven products	Admin maps coverages to policy types, so no product rules are hard-coded
D5	No separate Vehicle Service	Vehicles are always accessed through a customer; avoids unnecessary service calls
D6	Explicit lifecycle states	Each entity has defined states and owners, preventing invalid transitions

7. Technology Stack

Area	Technology
Frontend	Angular, TypeScript, TailwindCSS
Backend	ASP.NET Core Web API, C#
Data	SQL Server, Entity Framework Core, EF Core Migrations
Authentication	ASP.NET Identity, JWT, role-based authorization

Area	Technology
API style	REST over HTTPS, JSON
Testing	xUnit with Moq or NSubstitute; Jasmine/Karma for Angular; Postman or Swagger for API (<i>Confirm</i>)

8. Security Design

8.1 Authentication flow

Login → ASP.NET Identity validates credentials → JWT issued with user ID and role claims
 → Angular stores the token and attaches it through an HTTP interceptor → route guards check the role in the UI → API authorization attributes check the role again on the server
 → protected API executes.

8.2 Controls

Control	Implementation
Password storage	ASP.NET Identity hashing; plain text is never stored
Token lifetime	Short-lived access token plus refresh token
Authorization	Role attributes on controllers and policy checks on resource ownership
Ownership checks	A customer can access only their own profile, vehicles, policies and claims
Input validation	DTO validation and parameterized queries through EF Core
Transport	HTTPS only
File upload	Type and size validation; files stored outside the web root
Secrets	Connection strings and signing keys kept in configuration or a secret store, never in source control

The UI guard is a convenience only. The server is always the enforcement point.

9. Business Logic: Lifecycles and Rules

9.1 Quote lifecycle

From	To	Trigger	Owner
Created	Generated	Premium calculated from vehicle, coverage, owner age and city	System
Generated	Sent	Quote presented to the customer	System
Sent	Accepted	Customer accepts before ValidUntil	Customer
Sent	Rejected / Expired	Customer declines or validity passes	Customer / System

9.2 Policy lifecycle

From	To	Trigger	Owner
Pending	Issued	Underwriter approves the application	Underwriter
Issued	Active	Start date reached and payment recorded	System
Active	Expired	End date reached	System
Active	Cancelled	Customer cancels, refund calculated	Customer
Expired	Active (renewal)	Customer renews	Customer

9.3 Claim lifecycle

From	To	Trigger	Owner
Submitted	Assigned	Claim assigned to a Surveyor	Admin / System
Assigned	Surveyor Investigation	Surveyor starts inspection	Surveyor

From	To	Trigger	Owner
Surveyor Investigation	Recommendation	Findings and recommended amount submitted	Surveyor
Recommendation	Settlement	Decision recorded with approved amount	System
Submitted	Withdrawn	Customer withdraws before review	Customer

9.4 Business rules

- A claim can be filed only against an Active policy and within its coverage period.
- Premium is derived from the selected policy type's coverages, vehicle data, owner age and city. The Underwriter may override it, and the override is recorded.
- A policy is created only from an approved application.
- Policy types and coverages are read from configuration, so changing a product requires no deployment.
- Example product: Comprehensive EV = Battery + Flood + Theft + Roadside Assistance.

10. Data Design

10.1 Entities

Source column: **Original** means present in the earlier draft; **Proposed** means added to close a gap.

Entity	Purpose	Key attributes	Source
Users	Login identity (ASP.NET Identity)	UserId, Email (unique), PasswordHash, FullName, Phone, IsActive, CreatedAt	Original
Roles / UserRoles	Role catalogue and assignment	RoleId, Name; UserId + RoleId, AssignedAt	Original
Customers	Customer profile	CustomerId, UserId (unique FK), DateOfBirth, Address, Status	Original

Entity	Purpose	Key attributes	Source
Employees	Underwriters and Surveyors	EmployeeId, UserId (unique FK), EmployeeCode, EmployeeType, Department, IsActive	Original
EmployeeRequests	Admin approval of employee access	RequestId, UserId, RequestedType, Status, DecidedBy, DecidedAt	Proposed
Vehicles	Registered EV	VehicleId, CustomerId, Manufacturer, Model, RegistrationNumber (unique), ChassisNumber (unique), BatteryCapacityKwh, RegistrationYear, Odometer, Status	Original
PolicyTypes	Product tiers	PolicyTypeId, Name, Description, IsActive	Proposed
CoverageTypes	Protection elements	CoverageTypeId, Name, Description, IsActive	Proposed
PolicyTypeCoverages	Product mapping	PolicyTypeId, CoverageTypeId	Proposed
Quotes	Premium estimate	QuoteId, CustomerId, VehicleId, PolicyTypeId, OwnerAge, City, Idv, PremiumAmount, Status, ValidUntil	Original
Applications	Request to buy, underwriting record	ApplicationId, QuoteId, UnderwriterId (nullable), Status, UnderwritingRemarks, ReviewedAt	Proposed
Policies	Issued contract	PolicyId, ApplicationId (unique), CustomerId, VehicleId, PolicyNumber (unique), PremiumAmount, Status, StartDate, EndDate	Original

Entity	Purpose	Key attributes	Source
Claims	Claim record	ClaimId, PolicyId, CustomerId, VehicleId, AssignedSurveyorId (nullable), IncidentDate, IncidentType, Description, EstimatedAmount, ApprovedAmount (nullable), Status, CreatedAt	Original
ClaimSurveys	Surveyor findings	SurveyId, ClaimId, SurveyorEmployeeId, InspectionDate, Location, DamageDescription, RepairEstimate, RecommendedAmount, Recommendation, Remarks, Status	Original
Documents	Uploaded files	DocumentId, FileName, DocumentType, ReferenceType, ReferenceId, StoragePath, ContentType, UploadedByUserId, UploadedAt	Original
Notifications	Event messages and preferences	NotificationId, UserId, Template, Channel, Status, CreatedAt	Original (API)

10.2 Relationships

- One User has zero or one Customer profile and zero or one Employee profile; a User has many Roles.
- One Customer has many Vehicles, Quotes, Policies and Claims.
- One Vehicle has many Quotes; one Quote results in at most one Application and one Policy.
- One Policy has many Claims; one Claim has many Surveys and Documents.
- A PolicyType has many CoverageTypes through the PolicyTypeCoverages mapping.

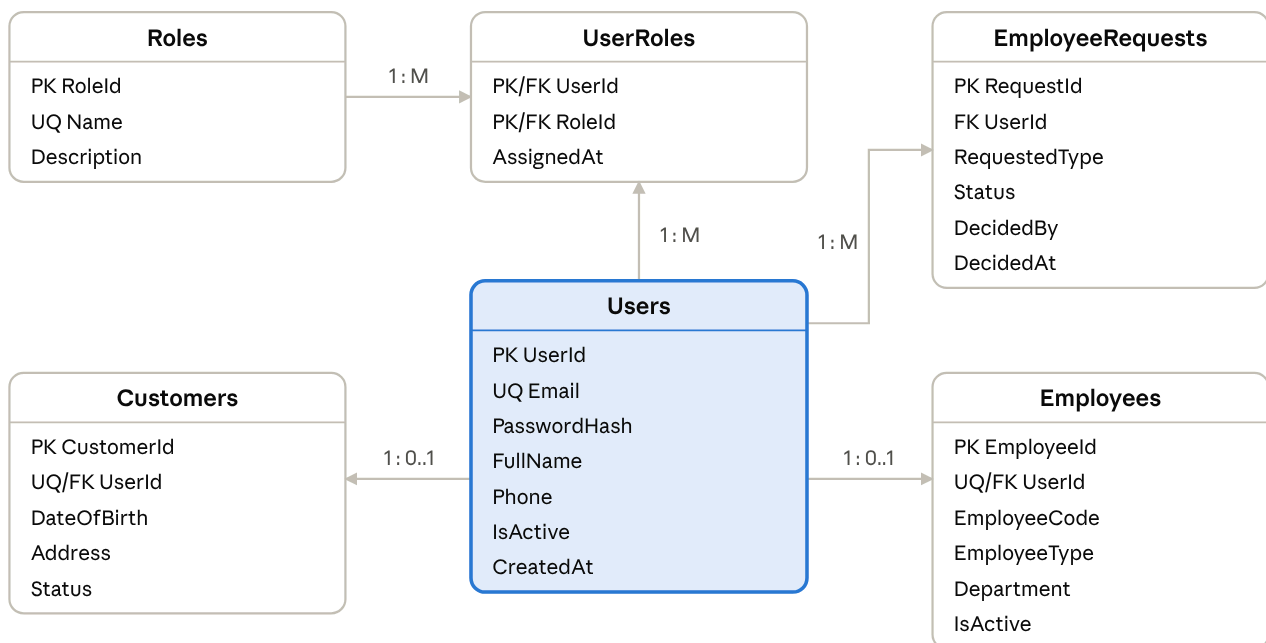
10.3 Data standards

- Primary keys are BIGINT identity columns (INT for Roles).
- Money uses DECIMAL(14,2); timestamps use DATETIME2 and are stored in UTC; dates use DATE.
- Status fields use a controlled set of values, enforced by enum mapping and check constraints.
- Documents use ReferenceType and ReferenceId (polymorphic link) so one table serves KYC, vehicle, policy and claim evidence.
- Schema changes are applied only through EF Core migrations.

10.4 Entity Relationship Diagrams

Three views show the reconciled schema. Arrows point from parent to child, and cardinality is written on each link.

Users hold roles and one customer or employee profile

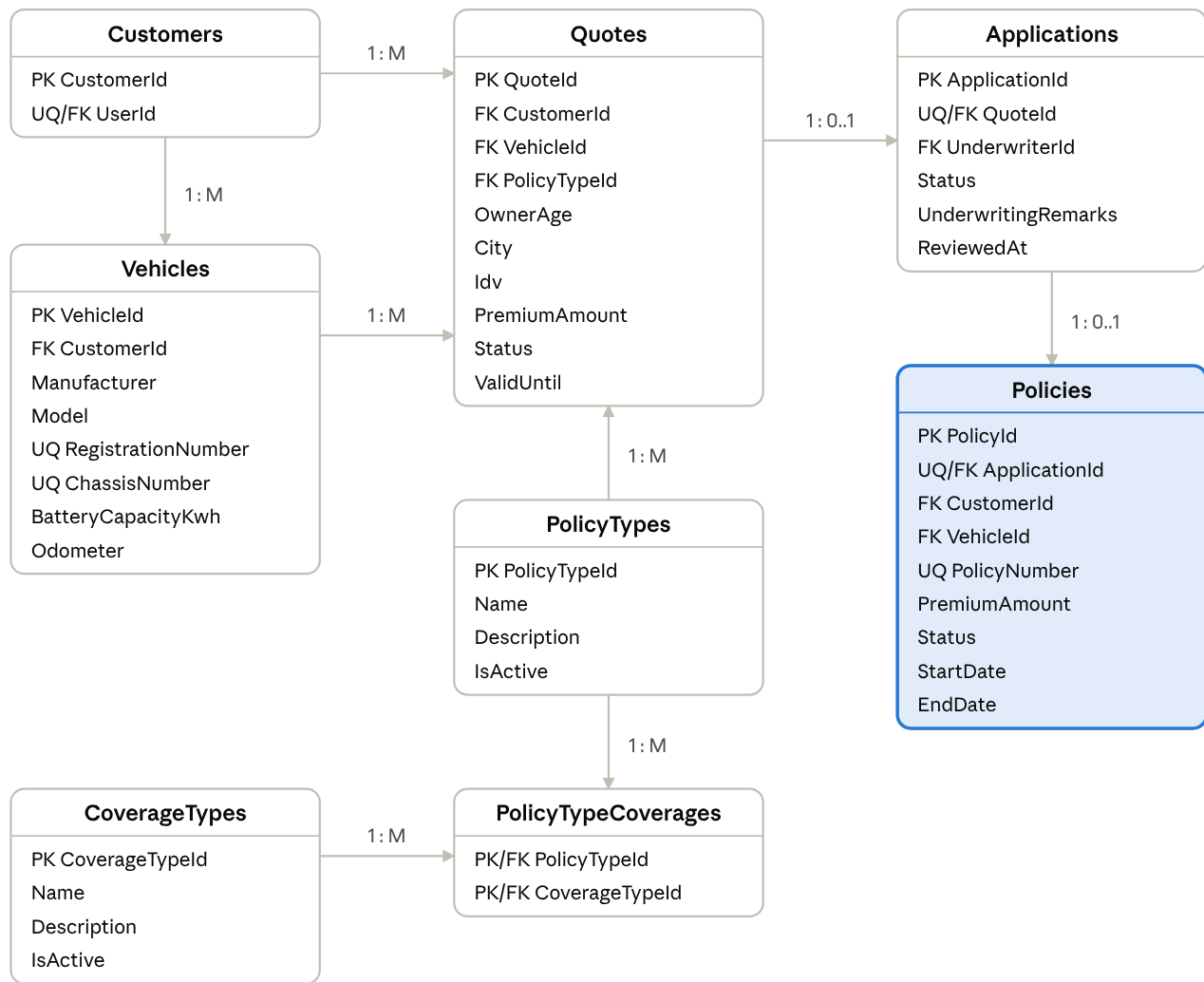


EmployeeRequests is a proposed entity; all others come from the original schema.
EmployeeType is Underwriter or Surveyor. PK primary key, FK foreign key, UQ unique.

ER diagram · users, roles, profiles

Figure 2. Identity and profiles view. One user can hold several roles and has at most one customer profile and one employee profile.

A policy is issued only from an application, which is built on a quote



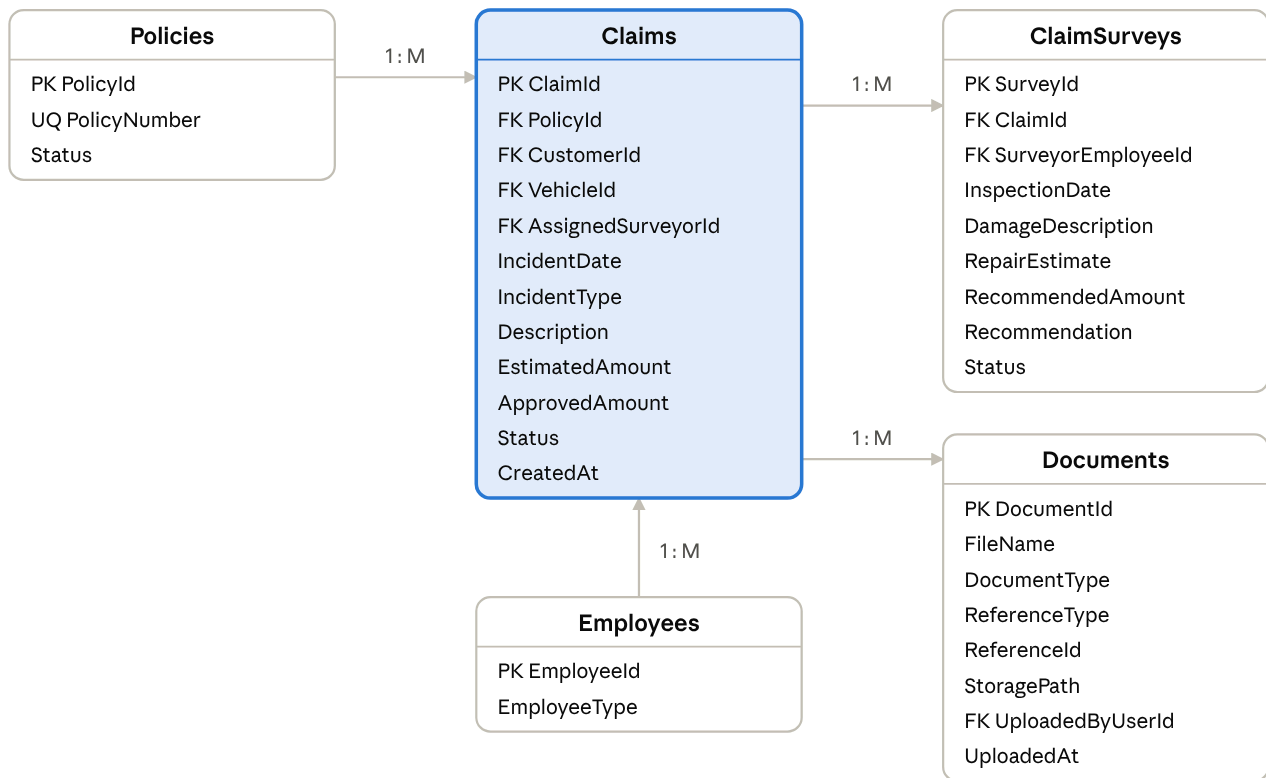
PolicyTypeCoverages is the join table that maps coverages to each policy type.

Quotes and Policies also carry CustomerId and VehicleId. UnderwriterId references Employees.

ER diagram · product, quote, application, policy

Figure 3. Product, quote and policy view. Admin-configured policy types drive quotes; an accepted quote becomes an application, and an approved application becomes one policy.

A claim collects survey reports and evidence before settlement



Documents link to a claim through ReferenceType and ReferenceId, not a database foreign key.

ClaimSurveys.SurveyorEmployeeId also references Employees. PK primary key, FK foreign key, UQ unique.

ER diagram · claims, surveys, documents

Figure 4. Claims view. A claim belongs to one policy and can have many survey reports and documents; the assigned Surveyor is an employee.

11. API Specification

Conventions. Base path `/api/v1` (*Proposed*). JSON request and response bodies.

Authorization: Bearer <token> on every endpoint except register, login and forgot-password. Standard codes: 200 OK, 201 Created, 400 validation error, 401 unauthenticated, 403 forbidden, 404 not found, 409 invalid state transition.

Error format:

```
{ "status": 409, "error": "InvalidStateTransition", "message": "Claim cannot
be filed on an expired policy.", "traceId": "..." }
```

11.1 Auth

Method	Endpoint	Purpose	Access
POST	/auth/register	Register customer	Public
POST	/auth/login	Log in, returns access and refresh tokens	Public
POST	/auth/refresh	Renew access token	Authenticated
POST	/auth/logout	Invalidate session	Authenticated
POST	/auth/forgot-password	Request reset token	Public
PUT	/auth/reset-password	Set new password with token	Public
PUT	/auth/change-password	Change password	Authenticated
GET	/auth/validate	Validate bearer token	Authenticated

11.2 Customers and vehicles

Method	Endpoint	Purpose	Access
POST	/customers	Create profile	Customer
GET	/customers/{id}	Fetch profile	Owner, Admin
PUT	/customers/{id}	Update phone and address	Owner
DELETE	/customers/{id}	Deactivate account	Owner, Admin
POST	/customers/{id}/kyc	Submit KYC document references	Owner
POST	/vehicles	Register EV	Customer
GET	/vehicles/{id}	Fetch EV	Owner, Admin
GET	/vehicles?customerId={id}	List customer EVs	Owner, Admin

Method	Endpoint	Purpose	Access
PUT	/vehicles/{id}	Update odometer, registration number	Owner
DELETE	/vehicles/{id}	Remove EV (soft delete)	Owner

Note: GET /customers/{id}/vehicles in the earlier draft duplicates GET /vehicles?customerId={id} . Keep one.

11.3 Quotes, applications and policies

Method	Endpoint	Purpose	Access
POST	/quotes	Generate premium estimate	Customer
GET	/quotes/{id}	Retrieve quote	Owner
POST	/applications	Apply for a policy from an accepted quote (<i>Proposed</i>)	Customer
GET	/applications?status=	List applications for review (<i>Proposed</i>)	Underwriter
PUT	/applications/{id}/premium	Adjust premium with remarks (<i>Proposed</i>)	Underwriter
PUT	/applications/{id}/decision	Approve or reject (<i>Proposed</i>)	Underwriter
POST	/policies	Issue policy after approval	System, Underwriter
GET	/policies/{id}	Fetch policy	Owner, staff
GET	/policies?customerId={id}	List customer policies	Owner, staff
PUT	/policies/{id}/renew	Renew policy	Customer
PUT	/policies/{id}/cancel	Cancel active policy, returns refund	Customer
GET	/policies/{id}/document	Download certificate	Owner
POST	/policies/{id}/endorsement	Mid-term change	Customer

The earlier draft used `/policies/quote`. Quotes are a separate resource, so `/quotes` is cleaner.

11.4 Claims

Method	Endpoint	Purpose	Access
POST	<code>/claims</code>	File claim	Customer
GET	<code>/claims/{id}</code>	Details and status	Owner, Surveyor, Admin
GET	<code>/claims?policyId={id}</code>	Policy claim history	Owner, Admin
POST	<code>/claims/{id}/documents</code>	Attach evidence	Owner
PUT	<code>/claims/{id}/assign</code>	Assign Surveyor (<i>Proposed</i>)	Admin
POST	<code>/claims/{id}/surveyor-report</code>	Submit findings and recommendation	Surveyor
PUT	<code>/claims/{id}/status</code>	Record decision, approved amount	System, Admin
DELETE	<code>/claims/{id}</code>	Withdraw before review	Owner

11.5 Admin (*Proposed*)

Method	Endpoint	Purpose
GET	<code>/admin/employee-requests</code>	List pending requests
PUT	<code>/admin/employee-requests/{id}</code>	Approve or reject
GET, POST	<code>/admin/policy-types</code>	List or create policy types
PUT, DELETE	<code>/admin/policy-types/{id}</code>	Update or deactivate
GET, POST	<code>/admin/coverage-types</code>	List or create coverage types
PUT	<code>/admin/policy-types/{id}/coverages</code>	Map coverages to a policy type

11.6 Documents and notifications

Method	Endpoint	Purpose
POST	/documents/upload	Store document (file, type, referenceId)
GET	/documents/{id}	Retrieve document
DELETE	/documents/{id}	Delete document
POST	/notifications/send	Create notification
GET	/notifications?userId={id}	Notification history
PUT	/notifications/preferences	Email and SMS opt-in

11.7 Sample exchange

Request: `POST /quotes`

```
{ "vehicleId": 1021, "policyTypeId": 1, "ownerAge": 32, "city": "Hyderabad" }
```

Response: `201 Created`

```
{ "quoteId": 5543, "premiumAmount": 18450.00, "validUntil": "2026-11-08T23:59:59Z" }
```

Publish the complete request and response schemas through Swagger/OpenAPI generated from the controllers, so the document and the code cannot drift.

12. Implementation Structure

12.1 Backend (*Confirm against repository*)

```

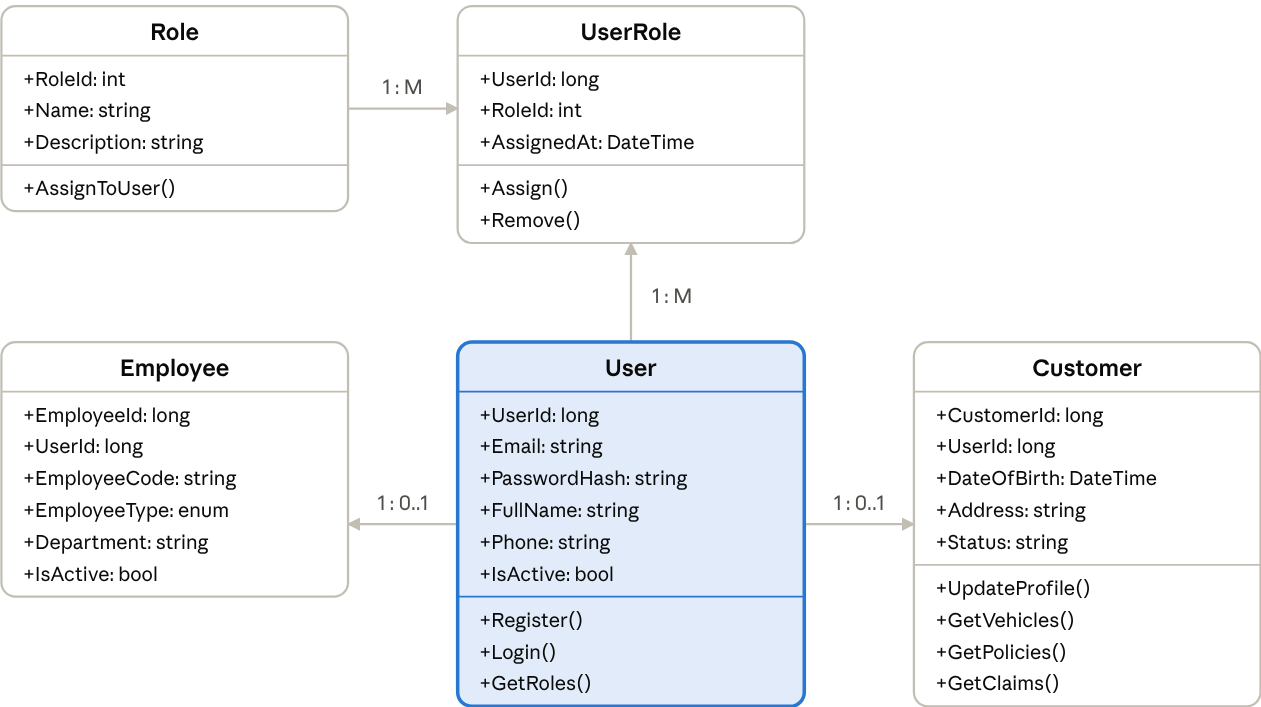
EvInsurance.sln
├─ EvInsurance.Api           Controllers, Program.cs, middleware, auth
                             setup
├─ EvInsurance.Application   Services, DTOs, interfaces, validators,
                             mapping
├─ EvInsurance.Domain        Entities, enums, state rules
├─ EvInsurance.Infrastructure DbContext, repositories, migrations, Identity,
                             file storage
└─ EvInsurance.Tests         Unit and integration tests
  
```

12.2 Key classes

Type	Examples
Controllers	AuthController, CustomersController, VehiclesController, QuotesController, PoliciesController, ClaimsController, AdminController
Services	AuthService, CustomerService, QuoteService, UnderwritingService, PolicyService, ClaimService, SurveyService
Entities	User, Role, Customer, Employee, Vehicle, Quote, Application, Policy, Claim, ClaimSurvey, Document
Employee types	Underwriter, Surveyor (the ClaimsOfficer type in the earlier draft is removed)

Four class diagrams follow: identity and profiles, the policy domain, the claims domain, and the controller, service and repository layering.

Roles are assigned to users, who hold a customer or employee profile

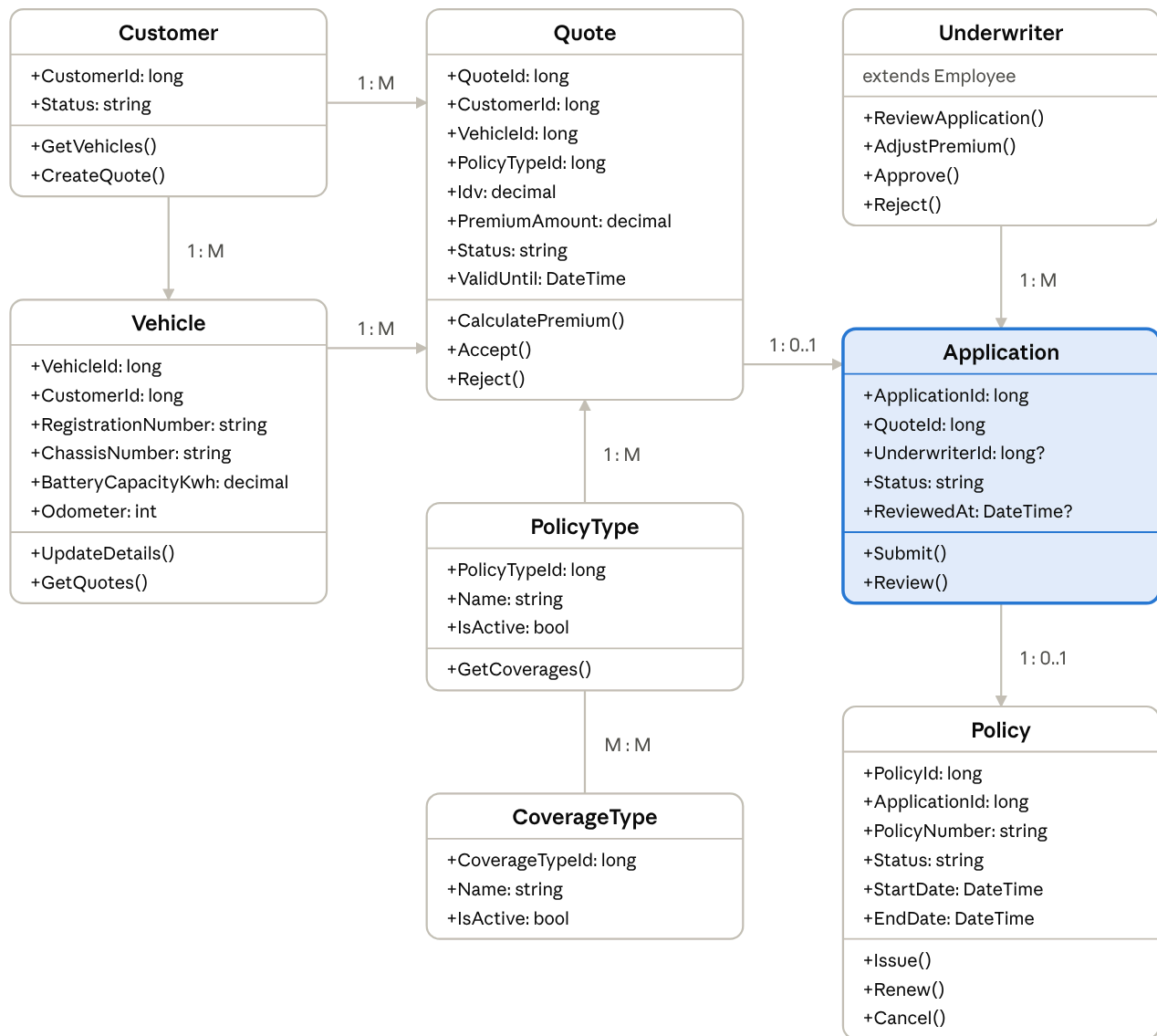


EmployeeType is Underwriter or Surveyor; each has its own class in the policy and claims diagrams.
Features are reached through the roles assigned in UserRole.

class diagram · identity and profiles

Figure 5. Identity and profile classes. A user can hold several roles and one customer or employee profile.

An underwriter reviews each application before a policy is issued

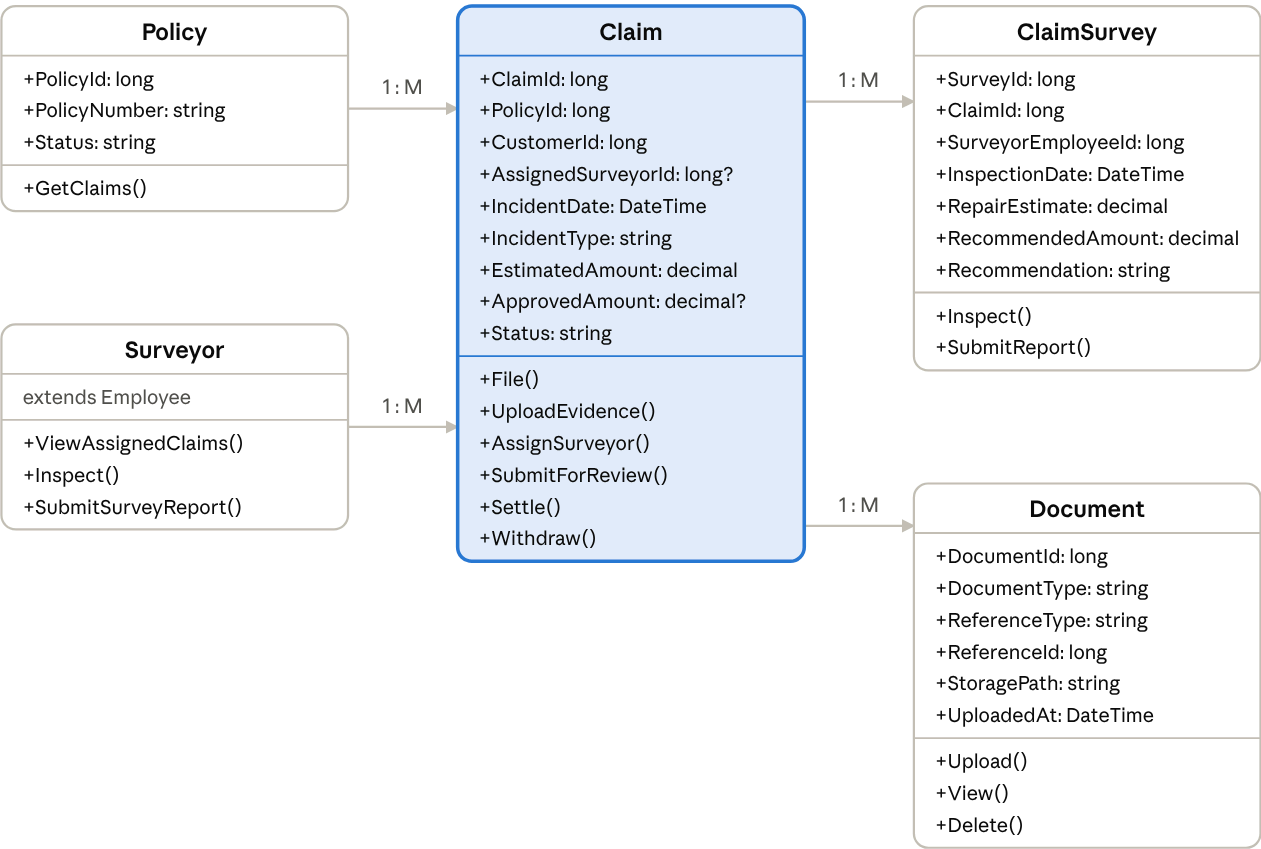


Policy also holds CustomerId and VehicleId. Admin maintains PolicyType and CoverageType.
Underwriter extends Employee with EmployeeType = Underwriter.

class diagram · quote, application, policy domain

Figure 6. Policy domain classes. A quote built on a policy type becomes an application; an Underwriter reviews it, and approval issues the policy.

The surveyor investigates each claim and recommends a settlement amount

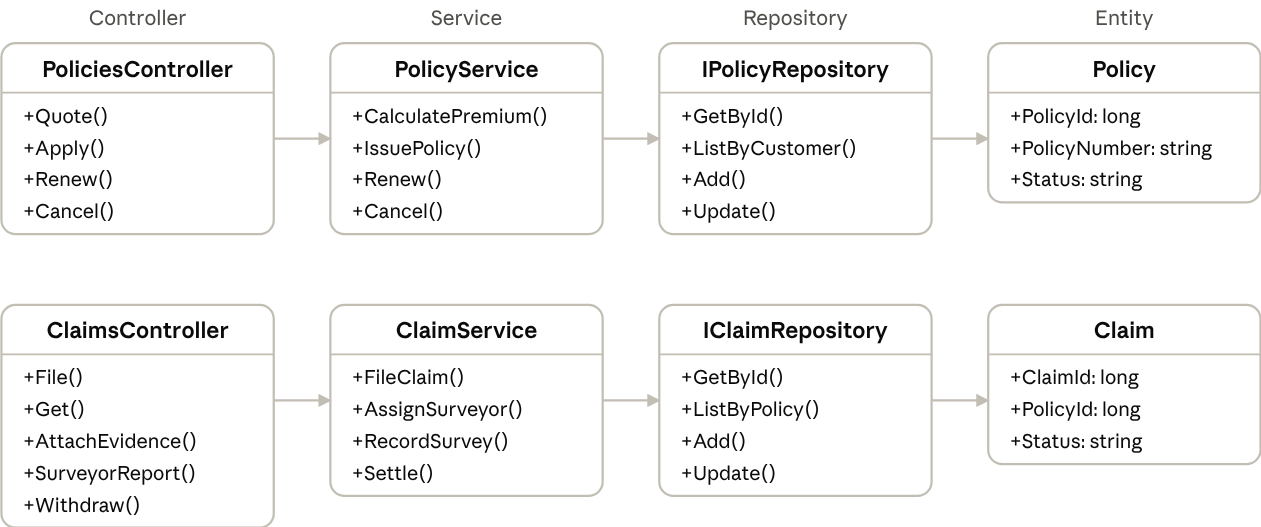


Documents link to a claim through ReferenceType and ReferenceId. A Surveyor submits the ClaimSurvey reports.
Surveyor extends Employee with EmployeeType = Surveyor.

class diagram · claims domain

Figure 7. Claims domain classes. A claim collects evidence documents and survey reports from the assigned Surveyor before it is settled.

Each module follows the same controller, service and repository pattern



Auth, Customer and Admin follow the same pattern. Repositories use ApplicationDbContext (EF Core).
Class names are indicative; confirm them against the repository.

class diagram · controller, service, repository layers

Figure 8. Layering for the Policy and Claims modules. Controllers call services, services use repository interfaces, and repositories persist the entities.

12.3 Frontend (*Confirm against repository*)

```
src/app
├─ core          auth service, JWT interceptor, route guards, models
├─ shared        reusable components, pipes, form controls
├─ features
│   └─ auth      login, register, reset password
│   └─ customer  dashboard, profile, EVs, quotes, policies, claims
│   └─ underwriter application queue, review, decision
│   └─ surveyor   assigned claims, inspection report
│   └─ admin      employee requests, policy types, coverage types, products
└─ app.routes.ts
```

Concern	Approach
Routing	Lazy-loaded feature routes per role
Guards	authGuard (logged in) and roleGuard (role match)

Concern	Approach
HTTP	Interceptor adds the JWT and handles 401 by refreshing or redirecting to login
Styling	TailwindCSS utility classes; responsive layout
Forms	Reactive forms with validation messages

13. Testing Strategy

13.1 Approach

Level	Scope	Tooling
Unit	Premium calculation, state transitions, validators	xUnit, Moq
Service / integration	Services with an in-memory or test database	xUnit, EF Core test database
API	Endpoints, status codes, role enforcement	Postman collection or Swagger
UI	Components, guards, forms	Jasmine/Karma
End-to-end	Complete journey from registration to settlement	Manual script or Playwright (<i>Optional</i>)

13.2 Test case format

Every test case records: ID, requirement traced, title, precondition, steps, expected result, actual result, status (Pass/Fail), tester and date.

13.3 Sample test cases

ID	Req	Scenario	Expected result
TC-01	FR-01	Register with a new email	201; user created with Customer role

ID	Req	Scenario	Expected result
TC-02	FR-01	Register with an existing email	400 or 409 with a clear message
TC-03	FR-16	Customer calls an Underwriter endpoint	403 Forbidden
TC-04	FR-05	Register EV with a duplicate registration number	Rejected; unique constraint respected
TC-05	FR-06	Generate quote for a valid EV	Quote returned with premium and ValidUntil
TC-06	FR-08	Underwriter adjusts premium	New premium and remarks saved with actor and time
TC-07	FR-09	Approve application	Policy created with unique number, status Issued
TC-08	FR-11	File claim on an expired policy	409; claim not created
TC-09	FR-12	Surveyor submits report on an unassigned claim	403 Forbidden
TC-10	FR-14	Admin maps Battery coverage to Premium EV	Mapping saved; reflected in new quotes
TC-11	FR-16	Customer reads another customer's policy	403 or 404

14. Deployment and Operations

Aspect	Current	Planned
Hosting	Local development	Cloud hosting (<i>future</i>)
Packaging	Manual build	Docker images
Pipeline	Manual	CI/CD with build, test and deploy stages
Database	SQL Server with EF Core migrations	Managed SQL service with backups

Aspect	Current	Planned
Configuration	appsettings and environment variables	Secret store
Logging	Application logs	Centralized logging and monitoring

15. Risks and Open Issues

#	Item	Impact	Action
R1	Earlier draft described microservices with a Payment Service and per-service databases; the implementation is layered	Reviewers see conflicting designs	Confirm topology and update Section 6 if needed
R2	Payment handling is not defined in the deck	Gap in the policy issuance flow	Document whether payment is simulated, recorded or out of scope
R3	Application entity is referenced but missing from the earlier ER diagram	Incomplete data model	Add to ERD and confirm the schema
R4	Premium formula is not documented	Hard to test and explain	Add the rating formula and factors to Section 9
R5	No measured performance results	NFR-03 is unverified	Run a basic load test and record results

16. Results and Screenshots

Capture and insert the following with a one-line caption each.

#	Screen	Role
1	Login and registration	Customer
2	EV registration form	Customer

#	Screen	Role
3	Quote result	Customer
4	Application review queue and decision	Underwriter
5	Policy details and lifecycle status	Customer
6	Claim submission with evidence	Customer
7	Assigned claim and survey report	Surveyor
8	Employee requests, policy types, coverage types, product configuration	Admin
9	Test run summary (unit and API)	Developer

17. Conclusion and Future Scope

The system delivers a complete, role-based EV insurance journey from registration to claim settlement, with EV-specific data and configurable coverages built into the domain. Explicit lifecycle states, layered architecture and server-side authorization make it maintainable and secure.

Future scope: Docker and CI/CD, cloud deployment, real external APIs (payment, KYC, notifications), asynchronous messaging for events such as claim assignment, and advanced EV risk calculation using battery health, usage and charging behavior.

Appendix A: Inconsistencies Found in the Earlier Draft

Area	Earlier draft	This document
Architecture	Five microservices with separate databases, service registry and gateway	Layered Clean Architecture with five modules; confirm topology (R1)
Roles	Customer, Insurance Employee, Administrator; class diagram includes ClaimsOfficer	Customer, Underwriter, Surveyor, Admin; no Claims Officer

Area	Earlier draft	This document
Quotes	ER and table differ (Underwriter, Idv, Status vs OwnerAge, City, ValidUntil)	Unified attribute set in 10.1
Policies	Table uses ApplicationId; ER uses Quoteld; no Applications table	Application entity added
Vehicles	ER has Manufacturer and ChassisNumber; table has RegistrationYear and Status	All merged
Customers	Table has FullName; ER does not	FullName kept on Users only, to avoid duplication
Documents	ER uses separate ClaimId, PolicyId, VehicleId; table uses ReferenceType and Referenceld	Polymorphic reference kept
Claims	Table has ApprovedAmount; ER splits it into ClaimReviews; no ClaimSurveys table	ApprovedAmount on Claims; ClaimSurveys added; ClaimReviews removed
API	Missing underwriter, admin and assignment endpoints; mixed quote path	Added and normalized
Sections	8.1, 8.4, 9, 10 and 11 empty; section 9 missing	Completed or templated

Appendix B: Pre-Submission Checklist

- Confirm every (*Confirm*) and (*Proposed*) item against the repository.
- Replace the sample folder structures with the real ones.
- Check the architecture, ER and class diagrams against the final schema and code, and update them if the implementation differs.
- Add screenshots and actual test results.
- Export Swagger/OpenAPI and attach it as the API reference.